Vysion XR — Software Requirements Specification

ExecuTorch Hackathon · Hacker//Squad · Oct 17–18, 2026 · San Francisco

Oct 17, 2026 · @Isaias Hernandez

1. Purpose, scope, and judging alignment

Vysion XR is a native Android XR app for Samsung Galaxy XR with three modes: Text Reader, Audio Description, and Navigation. All AI inference runs on the headset through ExecuTorch; nothing leaves the device. Five people build it in roughly 30 working hours across Saturday and Sunday.

The rubric puts 65 of 100 points on use case, working ExecuTorch integration, and on-device efficiency. Every requirement below is tagged with the rubric criterion it earns, and anything that earns none is cut.

Criterion	Pts	Judges' own words	What earns it in Vysion XR
Use case & impact	40	A useful idea, a clear problem, a strong experience	One headset, three real accessibility tasks, hands-free, voice in / spatial audio out
Technical implementation	25	A working application with ExecuTorch integration	Kotlin + Jetpack XR app; ExecuTorch runs the vision-language model and the detector on live passthrough frames
On-device AI & efficiency	15	Local processing, privacy, thoughtful resource use	Airplane-mode demo; live diagnostics panel showing ms/frame, memory, backend; no network permission in the manifest

Criterion	Pts	Judges' own words	What earns it in Vysion XR
Deployment & documentation	10	Clear setup instructions and an accessible application	Reproducible README, pinned ExecuTorch version, prebuilt APK + .pte models, TalkBack-tested UI
Presentation & demonstration	10	Show what works and explain it clearly	90-second live demo with a recorded backup; architecture slide; measured numbers, not claims

In scope: the three modes on Galaxy XR, on-device inference, voice command and TTS output, a diagnostics overlay, docs, and a demo. **Out of scope:** phone companion app, cloud anything, account/login, model training from scratch, multi-language beyond English and Spanish, and any safety claim for unassisted blind navigation.

Target quality bar: 85+/100. A project that only draws boxes on a video feed would land far lower, mostly on the 40-point use-case line.

2. Product overview

One headset, three spoken commands, two models. The user says "Vysion, read this", "Vysion, describe", or "Vysion, guide me" and hears an answer within seconds. The world-facing camera, depth, planes, and anchors all come from Android XR itself; the understanding comes from ExecuTorch running on the Snapdragon XR2+ Gen 2.

Mode	Trigger	Input	On-device model	Output
Text Reader	"read this" or pinch-and-hold	One world-camera frame, center crop	Vision-language model (VLM), read-verbatim prompt	TTS reads the text; large-print transcript panel; "repeat", "slower", "in Spanish"
Audio Description	"describe" or "what's around me"	One world-camera frame	Same VLM, describe prompt (3 sentences, hazards first, left to right)	TTS; follow-up questions reuse the same frame

Mode	Trigger	Input	On-device model	Output
Navigation	"guide me" or "find the door"	Continuous: depth texture + plane labels from ARCore for Jetpack XR, camera frames at 5 fps	Light object detector for people/obstacles; VLM one-shot to locate the door; native anchors to hold it	Spatial-audio pings from the target, short spoken cues ("door, 2 o'clock, 3 meters"), highlighted outline in passthrough

One-model strategy. Text Reader and Audio Description are the same VLM with different prompts, so the hardest export happens once. The detector is the only second model and exists because the VLM is too slow to run continuously.

Why a headset and not a phone. Hands stay free, the camera points where the head points, spatial audio can place a cue in the direction of the target, and anchors keep a target in memory when it leaves the view. The Galaxy S25 Ultra in the same track runs the identical ExecuTorch pipeline and is the fallback demo device if headset camera access fails.

3. Users and accessibility stance

Primary users are people with low vision and anyone in a situationally impaired moment: an unfamiliar building, poor light, hands full. Secondary users are Spanish/English learners who want signs and menus read and translated.

We claim an assistive aid, not a safety-critical device. The README, pitch, and app onboarding all say: "Vysion XR helps you read, understand, and move through a space. It is not validated for unassisted travel by blind users." That sentence protects the 40-point use-case line from the first hard question a judge asks.

Design rules that follow from the users:

- Every visual element has an audio twin; the app is fully usable with eyes closed.
- Voice first, gestures second, panel buttons third. No mode requires looking at a menu.
- Directions use clock-face bearings and meters ("door, 2 o'clock, 3 meters"), hazards are spoken before descriptions, and every utterance is interruptible with "stop".
- Transcript text is 48 dp or larger at 7:1 contrast; every control carries a TalkBack label.

- Nothing is spoken that was not measured or seen: no "probably a door".

4. Functional requirements

P0 must work for the demo; P1 is what separates us from a box-drawing project; P2 only after P1 is stable on the headset. Rubric column: UC = use case, TI = technical implementation, OD = on-device efficiency.

Global

ID	Requirement	Pri	Rubric
FR-G1	App launches on Galaxy XR into passthrough Full Space with one floating control panel and a spoken welcome that lists the three commands	P0	UC, TI
FR-G2	Voice commands via on-device SpeechRecognizer, started by pinch-and-hold or the panel mic button: read, describe, guide me, find the door, stop, repeat, slower, in Spanish	P0	UC
FR-G3	Mode changes are announced by TTS; the active mode is also shown on the panel	P0	UC
FR-G4	All speech output through Android TextToSpeech; "stop" interrupts within 300 ms	P0	UC
FR-G5	Diagnostics overlay (toggle): model name, backend, last inference ms, detector fps, app memory MB, "Network permission: none". Every value measured at runtime	P0	OD
FR-G6	Spanish output in all three modes (TTS voice + VLM prompt)	P1	UC
FR-G7	ExecuTorch TTS (Supertonic or Voxtral TTS) replaces Android TTS so speech out is also on ExecuTorch	P2	TI, OD

Text Reader

ID	Requirement	Pri	Rubric
FR-T1	On "read this", capture one world-camera frame, run the VLM with the read-verbatim prompt, and begin speaking within 6 s	P0	UC, TI
FR-T2	Show the transcript in a large-print spatial panel (48 dp+, 7:1 contrast) anchored where the user was looking	P0	UC
FR-T3	"Repeat" replays; "slower" replays at 0.7x rate	P1	UC
FR-T4	"In Spanish" re-prompts the VLM on the same frame for a translation	P1	UC
FR-T5	If head angular velocity is high at capture, say "hold still" and retry once	P1	UC
FR-T6	"Next" captures and reads the next page or sign without re-issuing the command	P2	UC

Audio Description

ID	Requirement	Pri	Rubric
FR-D1	On "describe", capture one frame, run the VLM with the describe prompt (3 sentences max, hazards first, left to right), begin speaking within 8 s	P0	UC, TI
FR-D2	A follow-up question ("what's on the table?") runs against the same frame without recapture	P1	UC
FR-D3	"Brief" and "detailed" switch prompt length	P1	UC
FR-D4	Pointing with the index finger at a region (hand tracking) describes only that region	P2	UC, TI

Navigation

ID	Requirement	Pri	Rubric
FR-N1	Proximity warning: when depth in the forward 60-degree cone falls below 1.0 m, play a rising tone and say "obstacle ahead"; repeat no more than every 2 s	P0	UC, OD
FR-N2	Detector runs at 5 fps or better on camera frames for person and common obstacles; detections drawn as outlines in passthrough	P0	TI, OD
FR-N3	"Find the door": VLM returns the door's horizontal position (left/center/right, or none); the app samples depth along that bearing, drops an anchor there, then speaks bearing and distance every 3 s with a spatial-audio ping from the anchor	P0	UC, TI
FR-N4	The door anchor persists when it leaves the field of view; guidance continues from head pose	P1	TI
FR-N5	Plane semantics: say "wall ahead" when a wall plane is within 1.5 m on the heading; say "floor ends" when the floor plane stops within 1.5 m	P1	UC
FR-N6	On arrival (anchor within 1 m), run the VLM once and speak handle side and whether the door is open	P1	UC
FR-N7	Multiple doors: "the one on the left" selects among candidates	P2	UC
FR-N8	Fine-tuned door class in the detector replaces the VLM one-shot for door finding	P2	TI

5. Non-functional requirements

Every number below is a target we measure and print on the diagnostics overlay; targets we miss are reported as misses, not hidden.

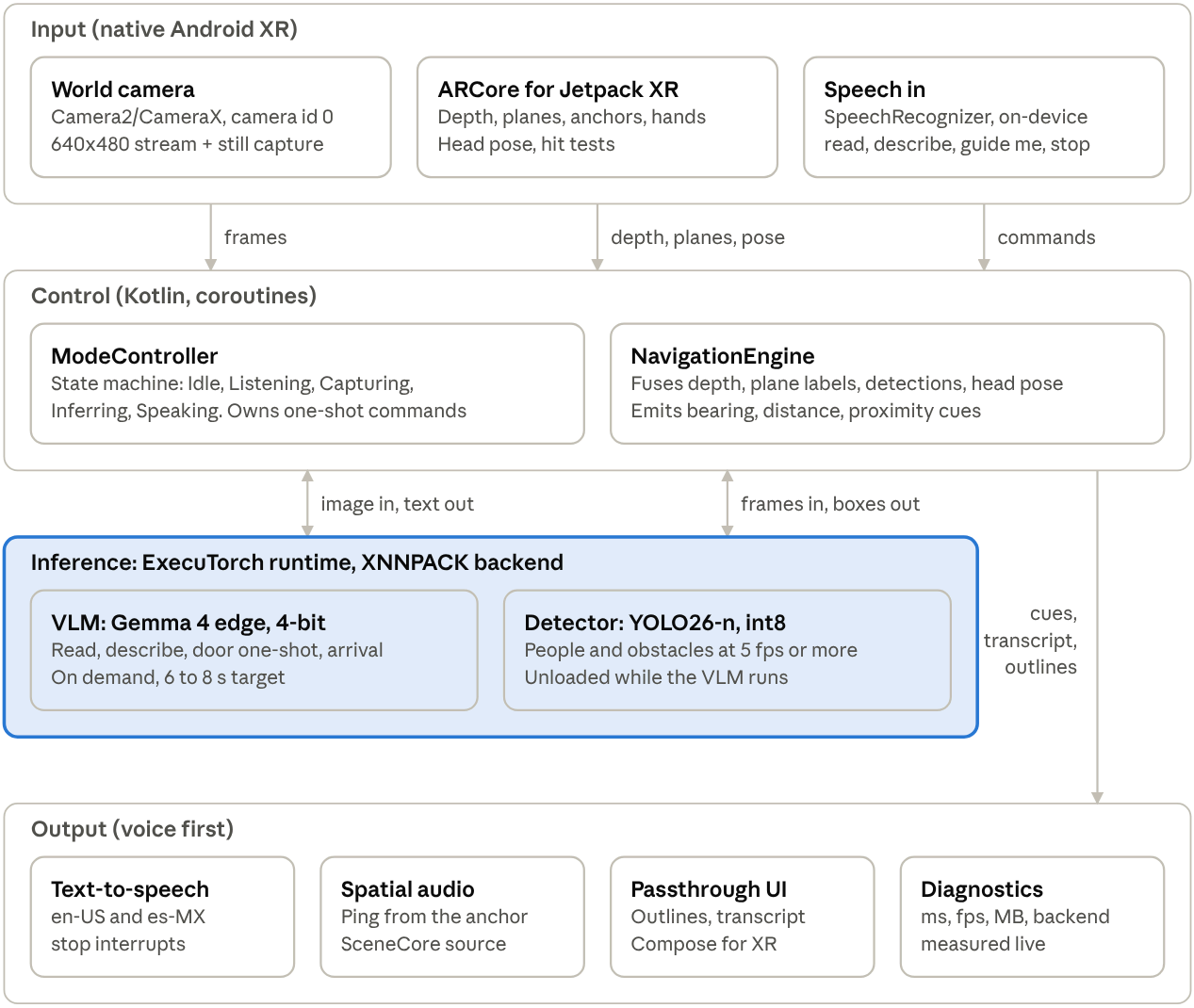
Area	Requirement	Target
Latency	Command recognized to action started	≤ 1.5 s

Area	Requirement	Target
Latency	Text Reader, capture to first spoken word	⩽ 6 s
Latency	Audio Description, capture to first spoken word	⩽ 8 s
Latency	Detector throughput during Navigation	⩾ 5 fps
Latency	Proximity warning after threshold crossed	⩽ 300 ms
Offline	Manifest declares no INTERNET permission; full demo runs in airplane mode	100% of features
Memory	App resident memory with VLM loaded	⩽ 4 GB
Memory	Detector unloaded while the VLM is generating, reloaded after	automatic
Thermal	10 min of continuous Navigation without detector dropping below 3 fps	pass
Privacy	Camera frames never written to storage; no analytics, no logging of frame content	enforced in code review
Accessibility	All features reachable by voice alone; transcript 48 dp+ at 7:1 contrast; TalkBack labels on every control	100%
Reliability	Any failed inference speaks an error within 2 s and returns to idle; the app is never silent for more than 10 s during a task	100%
Reproducibility	Fresh clone to running app on the headset using README only, with pinned ExecuTorch and Jetpack XR versions and a model download script	⩽ 30 min
Honesty	No hardcoded numbers on the overlay or in the pitch	100%

6. System architecture and external interfaces

Four layers in one Kotlin process: input adapters feed a mode controller, the controller calls two ExecuTorch modules, and output goes to speech, spatial audio, and passthrough overlays. No layer has a path to the network.

Nothing leaves the headset: four layers, two ExecuTorch models



architecture · 4 layers, 2 ExecuTorch models

Input comes from Android XR's own camera, perception, and speech stacks; Control decides what to run; the two ExecuTorch models answer; Output is speech first, visuals second. The only arrow that skips a layer is Control to Output, because inference results are spoken by Control, not by the models.

Interface	Library / version	Used for	Verify by
World-facing camera	Camera2 / CameraX, camera id 0, `CAMERA` permission (Google confirmation)	Frames for the VLM (on demand) and the detector (5 fps)	Sat, first hour on the headset
Depth, planes, anchors, hands, head pose	ARCore for Jetpack XR `androidx.xr.arcore:arcore:1.0.0-alpha15` (June 17, 2026); `SCENE_UNDERSTANDING_COARSE`, `SCENE_UNDERSTANDING_FINE`, `HAND_TRACKING` permissions	Proximity, wall/floor semantics, door anchor, pointing	Sat 13:00
Spatial UI and passthrough	Jetpack Compose for XR + SceneCore, Full Space with passthrough environment	Control panel, transcript panel, outlines, spatial audio point source	Sat 12:00
Inference	ExecuTorch Android AAR `org.pytorch:executorch-android` pinned to the current 1.3.x release; `Module` API for the detector, multimodal runner for the VLM; XNNPACK backend	Both models	Sat 15:00
Speech in	`android.speech.SpeechRecognizer`, on-device recognition preferred	Commands	Sat 14:00
Speech out	`android.speech.tts.TextToSpeech`, en-US and es-MX voices	Every spoken cue and transcript	Sat 14:00

Design decisions that keep five people from colliding:

- One `InferenceEngine` owns both models behind a `Mutex`; the detector is unloaded while the VLM generates and reloaded after, so peak memory is one model plus the runtime.
- Camera runs two streams: a 640x480 analysis stream for the detector and an on-demand still capture, center-cropped and resized to the VLM's input size.

- `ModeController` is a state machine: Idle, Listening, Capturing, Inferring, Speaking, back to Idle. Navigation is a background loop that any one-shot command can interrupt and that resumes afterward.
- Every module exposes a Kotlin `Flow` or a `suspend fun`; no module reaches into another's internals. The handoff table in section 8 is the contract.

7. Model plan

Two models ship; three more are stretch goals. The VLM is the whole product, so its export starts first and is measured on the headset before anything else is built on it.

Model	Role	Export path	Backend	Pri	Fallback
Gemma 4 edge multimodal (~2B)	Text Reader, Audio Description, door one-shot, arrival check	ExecuTorch `examples/models/gemma4` export, 4-bit weight quantization, Android multimodal runner	XNNPACK CPU	P0	SmolVLM-500M via the ExecuTorch Qualcomm VLM example if Gemma 4 is too slow or too large on the headset
YOLO26-n (COCO)	People and common obstacles during Navigation	ExecuTorch YOLO26 example, int8	XNNPACK CPU	P0	Detector off; proximity from depth alone
YOLO26-n door fine-tune	Continuous door tracking	Colab fine-tune on a public door dataset, same export	XNNPACK	P2	VLM one-shot + anchor (the P0 path)
Supertonic or Voxtral TTS	ExecuTorch speech out	ExecuTorch TTS examples	XNNPACK	P2	Android TTS
Qualcomm QNN delegate	NPU speedup for the VLM, sponsor relevance	QNN export of the same model, with on-site Qualcomm help	Hexagon NPU	P2	XNNPACK build stays the one we ship

Prompts live in one file, `PromptLibrary.kt`, and are quoted in the README:

- Read: "Read all text in this image exactly as written, top to bottom, left to right. Output only the text. If there is no text, say: No text found."
- Describe: "Describe this scene for someone who cannot see it, in at most three sentences. Hazards first, then what is on the left, center, and right. Do not guess."
- Door: "Is there a door in this image? Answer with one word: left, center, right, or none."

- Arrival: "Is this door open or closed, and which side is the handle on? One short sentence."

Measurement protocol: 20 runs per model on the headset in airplane mode; report median and p90 latency, peak memory, and model file size on the overlay and in the README. Numbers come from the device, never from a laptop.

8. Team of five: roles and tasks

Five owners, five modules, one integration branch. Each person owns a module end to end and ships against the handoff times below; nobody waits on anyone until the first merge at Sat 15:00. Put names in the Owner column.

Role	Owner	Owns	Hands off	Due
R1 XR Platform		App shell, passthrough Full Space, permissions, camera pipeline, panels, outlines, mode state machine	`FrameSource` (analysis stream + still capture), `ModeController` skeleton, `OverlayRenderer`	Sat 13:00
R2 ML / ExecuTorch		Model exports, `.pte` files, `InferenceEngine`, prompts, latency and memory measurement	`InferenceEngine` with `describe(bitmap, prompt)` and `detect(bitmap)` running on the headset	Sat 16:00
R3 Spatial Perception		ARCore for Jetpack XR session, depth, planes, anchors, hands, `NavigationEngine`, spatial audio cues	`NavigationCues` flow (proximity, bearing, distance) and `AnchorStore`	Sat 19:00
R4 Voice & Accessibility UX		Speech in, TTS, command grammar, phrasing, earcons, large-print panel spec, TalkBack labels, user testing	`CommandBus` (parsed commands as a flow) and `Speaker` (queue, interrupt, rate, locale)	Sat 15:00

Role	Owner	Owns	Hands off	Due
R5 Integration, Docs & Demo		Repo, build scripts, model download, diagnostics overlay, S25 Ultra fallback build, README, benchmark table, backup video, pitch	Green `main` on the headset after every merge; README that a stranger can follow	continuous

R1 XR Platform (Kotlin, Compose for XR, SceneCore)

- [ ] P0 Project skeleton: Compose for XR activity, Full Space with passthrough, `CAMERA`, `RECORD_AUDIO`, `SCENE_UNDERSTANDING_*`, `HAND_TRACKING` permissions requested with spoken explanations. Sat 11:00
- [ ] P0 Confirm world-camera frames arrive via CameraX on the headset (camera id 0); log resolution, format, fps. Sat 12:00
- [ ] P0 `FrameSource`: 640x480 analysis stream to a `Flow<Bitmap>` plus `suspend fun captureStill(): Bitmap` center-cropped and resized to the VLM input size. Sat 13:00
- [ ] P0 `ModeController` state machine with a `Mutex` around inference; one-shot commands interrupt Navigation and resume it. Sat 15:00
- [ ] P0 Floating control panel (mic, mode, diagnostics toggle) and large-print transcript panel per R4's spec. Sat 18:00
- [ ] P0 `OverlayRenderer`: detection outlines and the door highlight drawn in passthrough from normalized boxes. Sat 21:00
- [ ] P1 Anchor the transcript panel where the user was looking at capture time. Sun 09:00
- [ ] P1 Pinch-and-hold as the push-to-talk gesture via hand tracking. Sun 10:00

R2 ML / ExecuTorch

- [ ] P0 Pin `org.pytorch:executorch-android` to the current 1.3.x release; load a sample `.pte` on the headset to confirm the runtime works. Sat 11:00
- [ ] P0 Export YOLO26-n (COCO) to `.pte` with XNNPACK, int8; run on headset; record ms/frame. Sat 13:00
- [ ] P0 Export Gemma 4 edge multimodal with 4-bit weights following the ExecuTorch `examples/models/gemma4` path; load with the Android multimodal runner; first image-to-text on the headset. Sat 16:00

- [ ] P0 `InferenceEngine`: `describe(bitmap, prompt): String`, `detect(bitmap): List<Detection>`, load/unload lifecycle, timing hooks for the overlay. Sat 16:00
- [ ] P0 `PromptLibrary.kt` with the four prompts in section 7; tune wording against 10 real photos from the venue. Sat 19:00
- [ ] P0 Benchmark script: 20 runs per model, median and p90 latency, peak memory, file size; results into the README table. Sun 10:00
- [ ] P1 If Gemma 4 misses the 6 s / 8 s targets, export SmolVLM-500M and A/B on the headset. Sat 22:00
- [ ] P2 Door-class fine-tune on Colab, re-export, swap into `detect()`. Sun 09:00
- [ ] P2 QNN export of the VLM with on-site Qualcomm help; keep XNNPACK as the shipping build. Sun 11:00

R3 Spatial Perception and Navigation

- [ ] P0 ARCore for Jetpack XR session (`arcore:1.0.0-alpha15`) created from the Compose for XR session; depth, plane, hand, and device-pose configs enabled. Sat 12:00
- [ ] P0 Proximity: sample the depth texture in the forward 60-degree cone at 10 Hz; emit `Proximity(distanceM)` when under 1.0 m with 2 s debounce. Sat 15:00
- [ ] P0 `AnchorStore`: given a bearing (left/center/right from the VLM) and a depth sample along it, create an anchor; expose bearing and distance from the current head pose. Sat 18:00
- [ ] P0 `NavigationCues` flow: proximity, door bearing in clock-face form, distance in meters rounded to 0.5. Sat 19:00
- [ ] P0 Spatial audio: a SceneCore point source at the anchor playing a ping every 3 s while guiding. Sat 21:00
- [ ] P1 Plane semantics: wall-ahead and floor-ends cues from plane labels within 1.5 m on the heading. Sun 09:00
- [ ] P1 Keep guiding when the door leaves the field of view; test by turning 180 degrees and back. Sun 10:00
- [ ] P2 Pointing: index-finger ray from hand tracking to a region for region-only describe. Sun 11:00

R4 Voice and Accessibility UX

- [ ] P0 `CommandBus`: `SpeechRecognizer` with on-device preference; grammar for read, describe, guide me, find the door, stop, repeat, slower, in Spanish; fuzzy matching so "read it" and "read that" both work. Sat 14:00

- [ ] P0 `Speaker`: TTS queue with interrupt-on-stop, rate control, en-US and es-MX voices, earcons for listening, thinking, done, error. Sat 15:00
- [ ] P0 Phrasing guide: clock-face bearings, meters, hazards first, no hedging words; every cue string in `Cues.kt`, nothing hardcoded elsewhere. Sat 16:00
- [ ] P0 Transcript panel spec for R1: 48 dp+ text, 7:1 contrast, auto-scroll synced to speech. Sat 16:00
- [ ] P0 Confirm offline speech recognition works in airplane mode on the headset; if not, pinch-to-talk and panel buttons are the P0 path and recognition becomes P1. Sat 14:00
- [ ] P1 Blindfold test with two teammates: 5 read, 5 describe, 3 guide tasks; log every confusing cue and fix wording. Sat 21:00
- [ ] P1 TalkBack labels on every control; verify with TalkBack on. Sun 09:00
- [ ] P2 ExecuTorch TTS (Supertonic or Voxtral TTS) behind the same `Speaker` interface. Sun 11:00

R5 Integration, Docs and Demo

- [ ] P0 Repo with `main` protected, feature branches per role, a 15-minute merge cadence from Sat 15:00; one person (R5) merges and reinstalls on the headset after every merge. Sat 10:30
- [ ] P0 `scripts/fetch_models.sh` that downloads or builds the `.pte` files to `app/src/main/assets`; `./gradlew installDebug` works from a fresh clone. Sat 14:00
- [ ] P0 Diagnostics overlay fed by `InferenceEngine` timing hooks and `ActivityManager` memory; shows backend, model, median ms, fps, MB, "Network permission: none". Sat 18:00
- [ ] P0 S25 Ultra build of the same app with a flat 2D UI, used as the fallback demo and for faster ML iteration. Sat 20:00
- [ ] P0 README: problem, demo video link, architecture (export this doc's diagram), setup, model table with measured numbers, prompts, what runs locally, limitations, troubleshooting. Sun 11:00
- [ ] P0 Recorded backup demo (headset screen capture + room camera) of all three modes. Sun 12:00
- [ ] P0 Pitch: 3 slides plus the live demo; the 90-second script in section 11 rehearsed twice. Sun 13:00
- [ ] P1 Test matrix: 3 lighting conditions, 2 door types, partially open door, person walking across; results in README. Sun 10:00

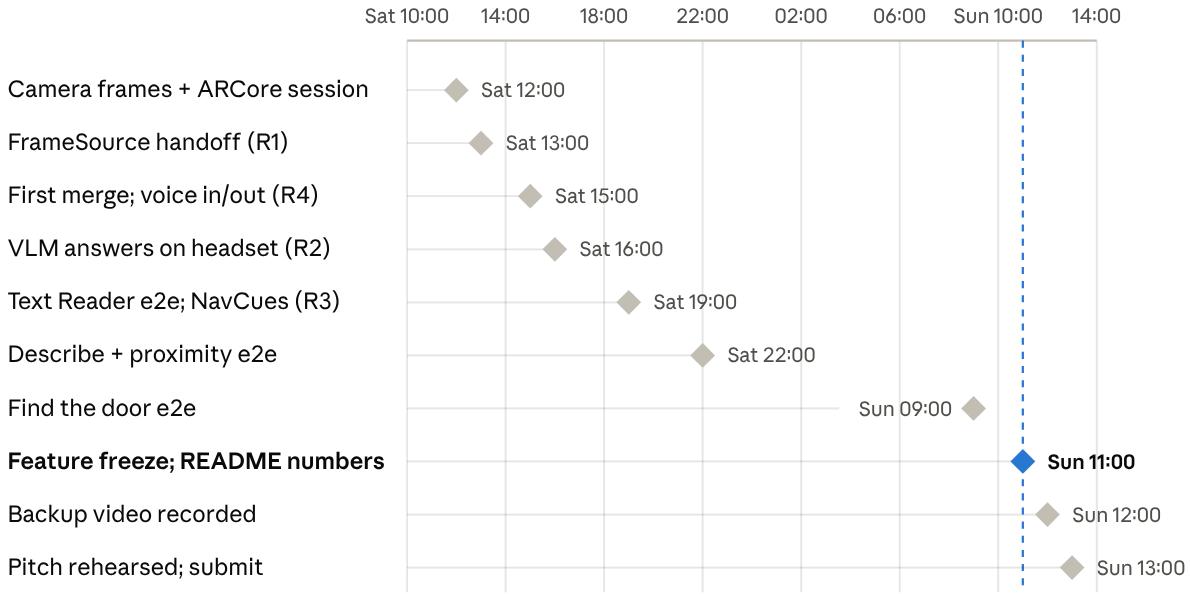
- [ ] P1 Submission form, team photo, and repo link ready before the deadline. Confirm the deadline with organizers Sat morning.

9. Schedule and integration gates

Ten gates on `main`, each one a thing that runs on the headset, not a slide. Saturday is for getting every module alive in isolation and merged by evening; Sunday morning is for the one feature that needs everything at once, Find the door; everything after the Sun 11:00 freeze is documentation, video, and rehearsal.

Feature freeze at Sun 11:00 leaves three hours for docs, video, pitch

Integration gates on main, Sat 10:00 to Sun 14:00. Confirm the judging time with organizers.



integration gates · Sat 10:00 to Sun 14:00

Each diamond is a merge to `main` that R5 installs and runs on the headset before anyone moves on. If a gate slips more than two hours, the P1 items for that role are cut first, then P2, never the gate.

10. Risks and mitigations

The two risks that can sink the project are camera access and VLM speed, and both get answered in the first six hours. Everything else has a cheaper fallback already in the plan.

Risk	Likelihood	Impact	Mitigation	Owner
World-camera frames not available to our app on this Galaxy XR build (permission denied, blank stream, unsupported format)	Medium	Fatal for Read and Describe on the headset	Test in hour one. Fallback: run Read and Describe on the S25 Ultra from the same track; keep Navigation on the headset using ARCore depth and planes only	R1
Gemma 4 edge too slow or too large on the XR2+ Gen 2	Medium	Misses 6 s / 8 s targets	Measure by Sat 16:00; switch to SmolVLM-500M; cap output tokens at 60; prefill once, answer follow-ups from cache	R2
Memory pressure with VLM and detector loaded	Medium	Crashes mid-demo	Unload the detector during VLM calls; 4 GB budget on the overlay; test 10 minutes continuous	R2, R5
Offline speech recognition unavailable on the headset	Medium	Voice commands need network	Pinch-to-talk plus panel buttons are P0; recognition becomes P1; never demo with Wi-Fi on	R4
ARCore depth noisy or missing	Low	Proximity cue unreliable	Fall back to plane hit tests and detector box height as a distance proxy; only speak distances we trust	R3

Risk	Likelihood	Impact	Mitigation	Owner
Door one-shot returns the wrong side	Medium	Guides the user to a wall	Ask the VLM twice on two frames and require agreement; otherwise say "I can't find a door, turn slowly"	R2, R3
Integration collapses Sunday morning	Medium	Nothing demos	15-minute merge cadence from Sat 15:00; R5 owns `main`; freeze at Sun 11:00 is not negotiable	R5
Headset tracking loss, thermal throttle, or battery during judging	Medium	Live demo fails	Recorded backup video; headset at 100% and cooled before our slot; demo under 90 s; S25 Ultra as the second device	R5
Overclaiming accessibility	Low	Judges push back on the 40-point line	The assistive-not-safety sentence appears in the pitch, README, and app onboarding	R4

11. Demo script, diagnostics panel, and submission checklist

The demo is 90 seconds, three modes, one airplane-mode moment, and measured numbers on screen the whole time. The person wearing the headset and the person speaking are two different teammates.

Demo script (90 s)

1. 0:00 Problem (speaker, 10 s): "Reading a sign, understanding a room, finding the exit. Three things a headset with cameras should do without sending a single frame to a server."

2. 0:10 Text Reader (20 s): wearer looks at a printed menu or sign and says "Vysion, read this." Spoken transcript plus the large-print panel on the mirror screen. Then "in Spanish."
3. 0:30 Audio Description (20 s): wearer turns to a cluttered table, says "Vysion, describe." Three sentences, hazards first.
4. 0:50 Navigation (25 s): wearer says "Vysion, find the door." Spatial ping, "door, 2 o'clock, 3 meters", wearer walks, outline follows, arrival cue with handle side.
5. 1:15 Proof (15 s): speaker holds up the headset's airplane-mode status on the mirror, points at the diagnostics panel: model, backend, median ms, fps, memory, "Network permission: none." One sentence on the architecture slide. Done.

Diagnostics panel fields (all measured at runtime)

Field	Source
Model and quantization	`InferenceEngine` metadata
Backend	ExecuTorch runtime delegate name
Last VLM latency, median of session	timing hooks around the runner call
Detector fps, 5 s window	frame counter in `detect()`
App memory, MB	`ActivityManager.getProcessMemoryInfo`
Network permission	read from the manifest at runtime, prints "none"

Submission checklist

- [ ] Repo public, `main` green, README complete with measured numbers, prompts, limitations
- [ ] `scripts/fetch_models.sh` tested from a fresh clone by someone who did not write it
- [ ] Prebuilt APK and `.pte` files attached to a release tag
- [ ] Demo video (backup) uploaded and linked from the README
- [ ] Architecture diagram exported from this doc into the README
- [ ] Three-slide pitch: problem, architecture, measured results
- [ ] Headset charged, airplane mode on, demo room scouted for a door, a sign, and a cluttered table
- [ ] Submission form filled before the deadline (confirm time and format with organizers Saturday morning)

- [ ] Team photo

12. Assumptions to verify before coding

Each of these is a yes/no we can answer in under an hour with the device in hand. Anyone whose module depends on an unverified assumption starts with the check, not the code.

- [ ] R1: CameraX delivers world-camera frames (camera id 0) to a third-party app on this Galaxy XR firmware with only the `CAMERA` permission; note resolution and format. Sat 11:00
- [ ] R1: Compose for XR Full Space keeps passthrough on while our panels are shown. Sat 11:00
- [ ] R2: The current `org.pytorch:executorch-android` 1.3.x AAR on Maven includes the multimodal runner and XNNPACK, so no from-source build is needed. Sat 11:00
- [ ] R2: Gemma 4 edge multimodal exports with the documented script and its `.pte` fits in memory on the headset; record file size and load time. Sat 14:00
- [ ] R3: ARCore for Jetpack XR `1.0.0-alpha15` depth and plane tracking return data on the headset with the `SCENE_UNDERSTANDING_*` permissions granted. Sat 12:00
- [ ] R3: SceneCore spatial audio point source is available and audible through the headset speakers. Sat 13:00
- [ ] R4: `SpeechRecognizer` works with network off on the headset. Sat 14:00
- [ ] R4: TTS has an es-MX or es-ES voice installed offline. Sat 14:00
- [ ] R5: We have both the Galaxy XR and the S25 Ultra from the track, ADB over Wi-Fi or USB-C to the headset works from each laptop, and the judging time and submission format are confirmed in writing from the organizers. Sat 10:30

Sources

- ExecuTorch Hackathon on Luma: tracks, devices, team size, prizes
- Judging rubric: 100 points across five criteria (photographed on site, Oct 17, 2026)
- Google confirms Android XR camera access and UploadVR on camera id 0 via Camera2/CameraX
- Android XR OpenXR extensions and permissions
- ARCore for Jetpack XR releases and overview
- ExecuTorch releases and examples index

- Using ExecuTorch on Android (Maven coordinates and backend list)